X ENCONTRO DE DESENVOLVIMENTO DE PROCESSOS AGROINDUSTRIAIS Uniube IFTM UFTM 2026

VERIFICAÇÃO DE CÓDIGO GERADO POR IA EM DEVSECOPS PARA SUPERVISÃO AGROINDUSTRIAL: ENSAIO COM BANDIT E ALOCAÇÃO DE HORAS PELO MÉTODO SIMPLEX

GERMANO AUGUSTO FREITAS BARBOSA[1*], ADRIANO DAWISON DE LIMA[1], LEANDRO AURELIANO DA SILVA[1]

[1]Universidade de Uberaba (UNIUBE), Curso de Bacharelado em Engenharia de Computação
*e-mail: germanofb@edu.uniube.br

RESUMO - Este trabalho combina revisão narrativa da literatura (2019–2026), um ensaio exploratório e um modelo ilustrativo de Programação Linear para discutir a verificação de código gerado por Inteligência Artificial Generativa (IAG) em DevSecOps, com contexto de aplicação proposto na camada de software de supervisão de uma unidade agroindustrial. O corpus reuniu onze documentos: dez estudos científicos, dos quais quatro são preprints, e um documento técnico institucional. No ensaio, cinco trechos em Python gerados por um assistente de IA, sem instruções de segurança, foram analisados com a ferramenta Bandit 1.9.4: quatro dos cinco trechos (80%) receberam ao menos um alerta automático, e uma falha de Path Traversal foi identificada apenas por revisão manual. Esses números descrevem o conjunto testado; não constituem taxa de vulnerabilidades confirmadas nem são comparáveis diretamente a outros estudos. Formulou-se ainda um modelo de Programação Linear, resolvido pelo Método Simplex e verificado computacionalmente, para alocar horas semanais da equipe entre análise estática e revisão humana. Com coeficientes hipotéticos, obteve-se $x_1 = 20$ h, $x_2 = 20$ h e $Z = 200$ pontos; as restrições de equipe e de revisão são ativas, e o pipeline tem folga de 4 h. O modelo é ilustrativo e depende de calibração empírica.

Palavras-chave: Inteligência Artificial Generativa. DevSecOps. Análise estática de código. Programação Linear. Método Simplex.

INTRODUÇÃO

O desenvolvimento de software contemporâneo é marcado por dois movimentos convergentes: de um lado, a consolidação da cultura DevSecOps, que incorpora práticas de segurança de forma contínua ao ciclo de vida de desenvolvimento; de outro, a expansão de modelos de Inteligência Artificial Generativa (IAG), sobretudo *Large Language Models* (LLMs), aplicados à geração, revisão e correção de código-fonte. Ferramentas de IA passaram a ser integradas a pipelines de Integração e Entrega Contínuas (CI/CD), tanto na geração de código quanto na triagem de vulnerabilidades e na sugestão de correções (Du et al., 2026; Firouzi e Ghafari, 2026).

A digitalização também alcança a agroindústria. Sistemas de supervisão e controle — *Supervisory Control and Data Acquisition* (SCADA), Controladores Lógicos Programáveis (CLPs) e dispositivos de Internet das Coisas (IoT) — sustentam o monitoramento de silos, secadores e linhas de processamento, e parte do software associado pode vir a ser escrita ou revisada com apoio de IAG. Pinheiro (2019) aponta que a convergência entre redes corporativas (TI) e industriais (TO) amplia a superfície de ataque de plantas automatizadas, e Zhang et al. (2024) analisam a (Gen)IA como instrumento de ataque e de defesa em sistemas de controle industrial. Contudo, a literatura também

mostra que o código produzido por assistentes de IA nem sempre é seguro (Pearce et al., 2021; Perry et al., 2023) e que os próprios modelos trazem riscos de cadeia de suprimentos, como o envenenamento de dados de treinamento e *backdoors* persistentes (Carlini et al., 2024; Hubinger et al., 2024).

Para delimitar o alcance, este trabalho adota como sistema de referência a camada de aplicação, escrita em Python, de um sistema de supervisão de uma unidade de armazenagem e secagem de grãos. Trata-se de um cenário de aplicação proposto pelos autores, e não de uma planta real estudada. Nessa camada consideram-se cinco componentes, que correspondem às cinco tarefas do ensaio: autenticação de operadores, armazenamento de credenciais, recebimento de arquivos (laudos e planilhas de lote), diagnóstico de conectividade com equipamentos de campo e leitura de arquivos de configuração em XML. Os riscos analisados são as classes de fraqueza associadas a essas tarefas — injeção de SQL (CWE-89), algoritmo criptográfico fraco (CWE-327), *Path Traversal* (CWE-22), injeção de comando do sistema operacional (CWE-78) e processamento de XML não confiável — e, apenas no plano da revisão, os riscos da cadeia de suprimentos de modelos. Ficam fora do escopo a programação de CLPs, os protocolos e redes de TO e a avaliação de segurança de uma instalação industrial.

O objetivo geral é examinar como verificações automatizadas e humanas podem ser combinadas na verificação de código gerado por IAG no contexto delimitado, por meio de três entregas distintas: (i) uma revisão narrativa da literatura publicada entre 2019 e 2026; (ii) um ensaio exploratório com cinco trechos de código gerados por IA e analisados com a ferramenta Bandit; e (iii) um modelo ilustrativo de Programação Linear (PL), resolvido pelo Método Simplex, para a alocação de horas de verificação. Os objetivos específicos são: a) sintetizar evidências sobre a segurança de código gerado por IA, o uso de LLMs na detecção e triagem de vulnerabilidades e os riscos da cadeia de suprimentos de modelos, descrevendo o desenho e a métrica de cada estudo; b) descrever o ensaio, separando alertas automáticos, achados manuais, confirmações e mitigações propostas; c) formular, resolver e verificar computacionalmente o modelo de PL, com análise de sensibilidade das capacidades; e d) discutir implicações e limites para o contexto delimitado. O ensaio não se destina a estimar taxas de vulnerabilidade, e o modelo não é calibrado empiricamente; ambos têm caráter demonstrativo.

MATERIAL E MÉTODOS

Trata-se de estudo exploratório que combina três procedimentos: revisão narrativa da literatura, ensaio experimental de pequena escala com análise quantitativa descritiva e modelagem matemática por PL. A revisão não seguiu protocolo de revisão sistemática; sua função é contextualizar o ensaio e o modelo.

Revisão da Literatura

A busca foi realizada nas bases IEEE Xplore, ACM Digital Library e arXiv e no portal do OWASP GenAI Security Project, complementada por literatura técnica sobre segurança cibernética industrial, em [datas das buscas — a informar]. Foram usados descritores em português e inglês — "IA Generativa"/"generative AI", "DevSecOps", "geração de código seguro"/"secure code generation", "detecção de vulnerabilidades"/"vulnerability detection", "cadeia de suprimentos de IA"/"AI supply chain", "envenenamento de dados"/"data poisoning" e "sistemas de controle industrial"/"industrial control systems" —, combinados pelos operadores booleanos AND e OR. [Expressões exatas por base e número de registros recuperados, triados e incluídos — a informar.]

Os critérios de inclusão foram: (a) publicação entre 2019 e 2026; (b) aderência a pelo menos um dos eixos — segurança de código gerado por IA, uso de IA na detecção ou triagem de vulnerabilidades, riscos da cadeia de suprimentos de modelos ou segurança cibernética de sistemas industriais; e

(c) enquadramento em uma de três categorias: C1, publicações revisadas por pares em periódicos ou anais; C2, preprints, admitidos por tratarem de tema emergente, identificados como não revisados por pares e com a versão consultada registrada; C3, documentos técnicos de organizações de referência. Foram excluídos duplicatas, conteúdos promocionais de fornecedores sem descrição metodológica e documentos fora dos eixos. Referências de apoio metodológico (Taha, 2017) e documentação de ferramentas e taxonomias (PyCQA, 2026; Python Software Foundation, 2026; MITRE Corporation, 2026) não integram o corpus.

O corpus final reúne onze documentos (Tabela 1): cinco artigos em periódico ou anais revisados por pares (C1); um trabalho técnico em anais de evento, cuja revisão por pares não pôde ser verificada (Pinheiro, 2019); quatro preprints (C2); e um documento técnico institucional (C3). A análise foi feita por leitura integral e categorização temática em quatro eixos: segurança do código gerado, detecção e triagem com IA, cadeia de suprimentos de modelos e segurança industrial.

Ensaio Exploratório com Geração de Código

Foram formulados cinco *prompts* em linguagem natural, um para cada componente delimitado — autenticação de usuário, armazenamento de senha, *upload* de arquivo, teste de conectividade via linha de comando (*ping*) e leitura de configuração em XML —, sem instruções explícitas de segurança, de modo a aproximar o uso cotidiano descrito por Perry et al. (2023). O código foi gerado em Python pelo assistente [nome e versão do modelo — a informar], em [data — a informar], [parâmetros de geração e número de gerações por prompt — a informar]. Os trechos foram analisados com a ferramenta de análise estática de segurança (SAST) Bandit, versão 1.9.4, em Python [versão — a informar], [comando e configuração utilizados — a informar], e inspecionados manualmente pelos autores. Os *prompts*, os códigos e os relatórios originais [serão disponibilizados] no repositório do trabalho (https://github.com/germanoafb/edepa-devsecops-simplex), que também contém um procedimento de reexecução do Bandit que preserva o relatório bruto em JSON.

Para conferir o mapeamento entre teste, severidade, confiança e CWE do Bandit 1.9.4, a ferramenta foi executada também sobre trechos sintéticos de controle, escritos para esse fim e disponíveis no repositório; esses trechos não substituem os códigos do ensaio. Na análise, distinguiram-se: (i) alerta automático, isto é, o registro emitido pelo Bandit, com teste, severidade e confiança próprios; (ii) vulnerabilidade confirmada, cuja presença e possibilidade de exploração foram verificadas por análise humana documentada; (iii) falso positivo, quando o alerta é refutado por análise humana documentada; (iv) risco não confirmado, ainda não verificado; e (v) mitigação proposta, que não constitui detecção. As proporções são apresentadas com denominador explícito.

Modelo de Programação Linear

Considera-se uma equipe de DevSecOps que verifica o código gerado por IA destinado ao sistema delimitado. As variáveis de decisão são x_1, horas semanais da equipe dedicadas às atividades de análise estática automatizada (configuração, execução no pipeline e triagem dos alertas), e x_2, horas semanais da equipe dedicadas à revisão humana do código. Ambas são medidas em horas de trabalho da equipe (h/semana), o que torna homogênea a soma na restrição de capacidade total. A restrição do pipeline expressa, na mesma unidade, o limite de atividades de SAST suportado pela infraestrutura de CI/CD, admitindo-se que cada hora de execução no pipeline corresponda a uma hora de acompanhamento e triagem pela equipe. Se x_1 representasse tempo de máquina, a Equação 2 deveria ser reescrita como $\alpha x_1 + x_2 \leq 40$, em que α é a fração de hora humana por hora de execução; o modelo adotado corresponde a $\alpha = 1$.

A função objetivo Z é um índice hipotético de redução de risco, em pontos por

semana. Os coeficientes 4 e 6 pontos/h e as capacidades de 40, 24 e 20 h/semana são hipóteses de um cenário ilustrativo: não foram calibrados com registros de tempo nem com uma escala de risco validada e não derivam quantitativamente dos cinco trechos do ensaio. A diferença entre os coeficientes traduz apenas a hipótese qualitativa de que a revisão humana alcança classes de falhas não cobertas pela análise estática, como ilustra o caso de *Path Traversal*. Admite-se que cada ponto represente risco tratado (detectado e corrigido), e não apenas detectado, e que as duas atividades atuem sobre conjuntos disjuntos de falhas, sem dupla contagem. O modelo pressupõe linearidade, taxas marginais constantes, divisibilidade das horas e independência entre as atividades. Sua formulação é dada pelas Equações 1 a 5.

Maximizar:

$Z = 4x_1 + 6x_2$ (1)

Sujeito a:

$x_1 + x_2 \leq 40$ (equipe) (2)

$x_1 \leq 24$ (pipeline de CI/CD) (3)

$x_2 \leq 20$ (revisão humana) (4)

$x_1, x_2 \geq 0$ (5)

Na forma padrão, introduzem-se as variáveis de folga s_1, s_2 e s_3 (h/semana), uma para cada restrição das Equações 2 a 4. O modelo foi resolvido manualmente pelo Método Simplex tabular (Taha, 2017) e verificado por um script em Python 3, que usa apenas a biblioteca padrão e está disponível no repositório do trabalho (https:*github.com/germanoafb/edepa-devsecops-simplex). O script (a) reproduz o Simplex tabular em aritmética racional exata, (b) enumera todos os vértices da região viável e (c), quando o SciPy está instalado, resolve o modelo pelo* solver// HiGHS. Também executa três cenários de sensibilidade, cada um alterando uma única capacidade em 1 h, verifica automaticamente a viabilidade e os resultados esperados e grava as saídas em JSON e CSV.

RESULTADOS E DISCUSSÃO

A Tabela 1 apresenta o corpus, com o desenho e a métrica de cada estudo. Como as métricas diferem em natureza e denominador, os resultados não são agregados em uma taxa comum.

Tabela 1: Corpus da revisão (2019–2026): categoria, desenho e métrica de cada documento

Autor(es) / Ano	Cat.	Desenho do estudo	Métrica ou resultado reportado
Pearce et al. (2021)	C1*	GitHub Copilot em 89 cenários associados a CWEs do MITRE Top 25; 1.689 programas gerados	≈40% dos programas com ao menos uma vulnerabilidade
Perry et al. (2023)	C1	Estudo com usuários em tarefas de segurança, com e sem assistente (codex-davinci-002)	Grupo com IA escreveu código menos seguro e mais frequentemente o julgou seguro
Wang et al. (2024)	C2	Benchmark CodeSecEval (v1): 180 amostras de 44 tipos de vulnerabilidade; geração e reparo de código	LLMs frequentemente desconsideram a segurança; dificuldade varia por tipo de vulnerabilidade
Carlini et al. (2024)	C1	Ataques de envenenamento split-view e frontrunning em conjuntos de treinamento em escala web	Envenenar 0,01% do LAION-400M ou do COYO-700M custaria cerca de US$ 60 (estimativa dos autores)
Hubinger et al. (2024)	C2	LLMs com backdoor inserido deliberadamente (prova de conceito)	Backdoor persistiu a ajuste supervisionado, aprendizado por reforço e treinamento adversarial
Firouzi e Ghafari (2026)	C1	GPT-4o na detecção e correção de mau uso de APIs criptográficas; benchmarks e repositórios reais	F1 > 0,90 em dois benchmarks, com falsos positivos relevantes em um deles

Autor(es) / Ano	Cat.	Desenho do estudo	Métrica ou resultado reportado
Du et al. (2026)	C2	433 alarmes de ferramenta SAST corporativa (Tencent); técnicas que combinam LLM e análise estática	Eliminação de 94–98% dos falsos positivos, com alta revocação
Yang et al. (2025)	C2	Framework conceitual em três camadas para IAG na detecção de vulnerabilidades em redes 6G; estudo de caso com LLM	Sem métrica comparável no resumo; análise qualitativa
Pinheiro (2019)	Anais	Análise conceitual de segurança cibernética em redes industriais na Indústria 4.0	Qualitativo: segurança integrada de TI e TO como decisão estratégica
Zhang et al. (2024)	C1	Análise pela Cyber Kill Chain com simulações em cenários de sistemas de controle industrial	Qualitativo: (Gen)IA na automação de ataque e de defesa
OWASP Foundation (2025)	C3	Documento de referência OWASP Top 10 para aplicações de LLM	Categoria LLM03:2025 – Supply Chain

Fonte: elaborado pelos autores (2026). C1: revisado por pares; C2: preprint, não revisado por pares (versão consultada nas referências); C3: documento técnico institucional; Anais: trabalho técnico em anais, revisão por pares não verificada. *Versão consultada no arXiv; publicada posteriormente no IEEE S&P 2022.

Os estudos sobre geração de código não medem a mesma grandeza. Pearce et al. (2021) reportam a proporção de programas vulneráveis (≈40%) entre 1.689 programas gerados em 89 cenários desenhados para induzir CWEs específicas; Perry et al. (2023) comparam a segurança das soluções de participantes com e sem assistente; e Wang et al. (2024) avaliam LLMs em um benchmark de 180 amostras. Como diferem em linguagens, modelos, tarefas, critérios de vulnerabilidade e denominadores, esses resultados não são reduzidos a uma faixa geral de prevalência. Em conjunto, indicam que código gerado por IA pode conter vulnerabilidades e que o assistente pode aumentar a confiança indevida do desenvolvedor.

Quanto ao uso de IA na detecção e na triagem, Du et al. (2026) avaliaram, em 433 alarmes de uma ferramenta SAST corporativa da Tencent, técnicas que combinam LLMs e análise estática e que eliminaram 94–98% dos falsos positivos com alta revocação; o ambiente é industrial no sentido de produção de software, e não de automação industrial, e o estudo é um preprint. Firouzi e Ghafari (2026) mostram que o GPT-4o detecta e corrige mau uso de APIs criptográficas com F1 superior a 0,90 em dois benchmarks, mas com falsos positivos relevantes em um deles. Yang et al. (2025) propõem um framework conceitual em três camadas para redes 6G; sua transposição para pipelines de DevSecOps é uma interpretação deste trabalho. Esses resultados apoiam o uso da IAG como camada complementar de triagem, sem autorizar estimativas de ganho para o contexto delimitado; no ensaio aqui relatado, nenhuma triagem por LLM foi executada.

Na cadeia de suprimentos, Carlini et al. (2024) mostram que o envenenamento de conjuntos de treinamento em escala web é viável a baixo custo, e Hubinger et al. (2024) mostram, em modelos treinados deliberadamente com *backdoor*, que esse comportamento pode persistir a técnicas usuais de treinamento de segurança. O OWASP GenAI Security Project (OWASP Foundation, 2025) reconhece esses riscos na categoria LLM03:2025 – Supply Chain, que recomenda verificação de proveniência, inventário de componentes e avaliação de modelos de terceiros.

Aplicação ao Contexto Agroindustrial Delimitado

Os modelos que ampliam a capacidade de detecção são, eles próprios, artefatos sujeitos a riscos de cadeia de suprimentos. No contexto delimitado, essa tensão se manifesta na camada de supervisão, na qual uma falha de autenticação ou de injeção de comando poderia expor dados de lotes ou permitir a execução de comandos no servidor. Efeitos sobre o processo físico — parada de linha, perda de matéria-prima ou comprometimento da segurança dos alimentos — dependeriam da integração dessa camada com o controle, que

está fora do escopo deste trabalho (Pinheiro, 2019; Zhang et al., 2024).

A Figura 1 apresenta uma arquitetura proposta de pipeline, que organiza a verificação em etapas sucessivas: geração de código, análise estática no pipeline de CI/CD, revisão humana e, somente então, implantação. As etapas com contorno contínuo foram exercitadas no ensaio; a implantação e a camada de governança da cadeia de suprimentos, alinhada à categoria LLM03:2025 (OWASP Foundation, 2025), são propostas e não foram implementadas nem avaliadas.

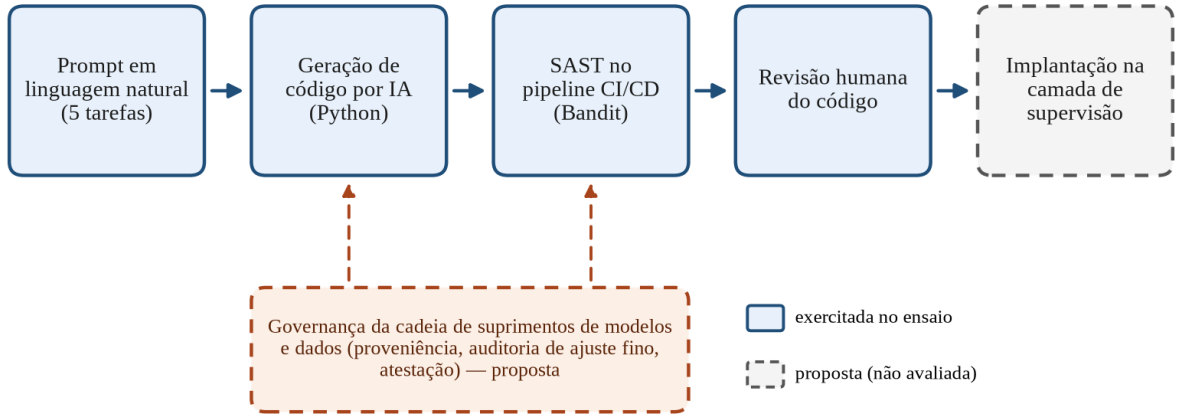


Figura 1: Arquitetura proposta de pipeline DevSecOps com IA generativa para o contexto delimitado

Ensaio Exploratório com o Bandit

A Tabela 2 resume os registros do ensaio, separando a classificação reportada pelo Bandit, o teste compatível na documentação da ferramenta e a situação após a análise humana.

Tabela 2: Registros do ensaio exploratório com o Bandit 1.9.4 e situação de cada achado

Trecho (tarefa)	**Registro do Bandit reportado (CWE do scanner; severidade / confiança)**	**Teste compatível no Bandit 1.9.4 (a confirmar no relatório)**	**Situação após análise humana**
1. Autenticação de usuário	SQL Injection por concatenação de string; CWE-89; Média / Baixa	B608 (confiança Baixa quando a consulta é montada fora da chamada execute)	Risco plausível; exploração depende da origem da entrada; confirmação não documentada
2. Armazenamento de senha	Hash criptograficamente fraco (MD5); CWE-327; Alta / Alta	B324 (hashlib)	Fraqueza verificável por inspeção; registro da confirmação pendente
3. Upload de arquivo	Nenhum alerta	— (não há teste geral de path traversal em operações de arquivo)	Achado manual: Path Traversal (CWE-22)
4. Teste de conectividade (ping)	Injeção de comando via shell; CWE-78; Alta / Alta	B602 ou B605; pode haver também B404 (Baixa / Alta)	Exploração depende de o host vir de entrada externa; confirmação não documentada
5. Leitura de configuração XML	Parsing de XML não confiável; CWE-20; “Baixa-Média / Alta” (agregado)	B405 (Baixa / Alta) e B314 (Média / Alta), se usado xml.etree	XXE (CWE-611) não confirmado; risco não confirmado

Fonte: elaborado pelos autores (2026). A segunda coluna reproduz os registros do ensaio; a terceira indica compatibilidade com a documentação (PyCQA, 2026) e com a execução de controle, e deve ser substituída pelos identificadores e linhas do relatório original.

Quatro dos cinco trechos (4/5 = 80%) receberam ao menos um alerta automático do Bandit. Essa proporção mede a cobertura automática do scanner sobre o conjunto testado e não equivale a 80% de vulnerabilidades confirmadas, pois a confirmação humana de cada alerta, com evidência do fluxo de entrada e da possibilidade de exploração, não está documentada para os quatro trechos. Os alertas também não são homogêneos: cada um tem severidade e confiança próprias, e a classificação "Baixa-Média / Alta" do trecho 5 agrega pelo menos dois registros, que devem ser apresentados separadamente com base no relatório original.

Separadamente, a revisão manual identificou no trecho de *upload* uma fraqueza de *Path Traversal* (CWE-22): o nome do arquivo fornecido pelo usuário era concatenado ao diretório de destino sem sanitização, o que, em tese, permitiria gravar arquivos fora do diretório pretendido. Esse resultado é contabilizado em denominador próprio — um achado manual em cinco trechos — e não é somado à detecção automática. A ausência de alerta é compatível com o conjunto de testes do Bandit, que não inclui verificação geral de *path traversal* em operações de arquivo (PyCQA, 2026).

Nos trechos 1 e 4, a exploração depende de que o nome de usuário e o host provenham de entrada externa sem validação, o que deve ser demonstrado no código. No trecho 2, o uso de MD5 para senhas é verificável por inspeção: o Bandit o classifica como CWE-327, e uma classificação humana mais específica seria CWE-916, uso de *hash* de senha com esforço computacional insuficiente (MITRE Corporation, 2026). No trecho 5, o Bandit atribui CWE-20 aos alertas de XML, e a classificação humana como XXE (CWE-611) não decorre apenas do uso de um parser: no módulo xml.etree da biblioteca padrão, o Expat não acessa arquivos locais nem abre conexões de rede por padrão (Python Software Foundation, 2026). A confirmação exigiria identificar a biblioteca, as versões do Python e do Expat e o fluxo de entrada, além de um teste controlado de exploração. Por isso, o trecho 5 é tratado como risco não confirmado.

As mitigações correspondentes — consultas parametrizadas (trecho 1), funções de derivação de chave para senhas, como scrypt ou PBKDF2 (trecho 2), normalização do caminho com verificação de que ele permanece no diretório de destino (trecho 3), execução de comandos sem *shell*, com lista de argumentos e validação do host (trecho 4), e *parsers* endurecidos, como o defusedxml (trecho 5) — são propostas e não foram implementadas nem testadas neste trabalho.

O ensaio tem caráter exploratório e não é estatisticamente representativo: são cinco trechos, uma única ferramenta e variáveis de geração não controladas. Seus resultados não são comparáveis à proporção de Pearce et al. (2021), obtida com outro modelo, outras tarefas e outro critério de vulnerabilidade, e não demonstram triagem por LLM, a eficácia de um pipeline em produção nem a segurança de uma instalação agroindustrial. O valor do ensaio está em ilustrar, no conjunto testado, que análise estática e revisão humana capturam classes distintas de falhas.

Resolução do Modelo pelo Método Simplex

O quadro inicial (Tabela 3) tem base (s_1, s_2, s_3) e Z = 0. Na linha Z, o menor coeficiente é −6 (coluna x_2); pelo teste da razão mínima (40/1 = 40 na linha de s_1 e 20/1 = 20 na de s_3; indefinida para s_2), x_2 entra na base no lugar de s_3. Após o pivoteamento, a linha Z passa a [−4, 0, 0, 0, 6 | 120]. O novo teste da razão (20/1 = 20 na linha de s_1 e 24/1 = 24 na de s_2) indica a entrada de x_1 no lugar de s_1, e o pivoteamento conduz ao quadro ótimo (Tabela 4), sem coeficientes negativos na linha Z.

Tabela 3: Quadro inicial do Método Simplex

Base	x_1	x_2	s_1	s_2	s_3	Solução
s_1	1	1	1	0	0	40
s_2	1	0	0	1	0	24
s_3	0	1	0	0	1	20
Z	−4	−6	0	0	0	0

Fonte: elaborado pelos autores (2026).

Tabela 4: Quadro ótimo do Método Simplex

Base	x_1	x_2	s_1	s_2	s_3	Solução
x_1	1	0	1	0	−1	20
s_2	0	0	−1	1	1	4
x_2	0	1	0	0	1	20
Z	0	0	4	0	2	200

Fonte: elaborado pelos autores (2026).

A solução ótima é $x_1^* = 20$ h/semana, $x_2^* = 20$ h/semana e $Z^* = 200$ pontos/semana, com folgas $s_1 = 0$, $s_2 = 4$ e $s_3 = 0$ h/semana. O resultado pode ser verificado diretamente: como $Z = 4(x_1 + x_2) + 2x_2 \leq 4 \times 40 + 2 \times 20 = 200$, nenhuma solução viável supera 200, valor atingido em (20, 20). O script reproduziu exatamente as Tabelas 3 e 4; a enumeração dos cinco vértices da região viável confirmou o ótimo; e o *solver* HiGHS retornou status ótimo com os mesmos valores. A Tabela 5 reúne o cenário-base e os três cenários de sensibilidade.

Tabela 5: Cenário-base e sensibilidade das capacidades (uma alteração por vez)

Cenário	b_1; b_2; b_3 (h)	x_1^*	x_2^*	Z^*	s_1; s_2; s_3
Base	40; 24; 20	20	20	200	0; 4; 0
Equipe +1 h	41; 24; 20	21	20	204	0; 3; 0
Revisão +1 h	40; 24; 21	19	21	202	0; 5; 0
Pipeline +1 h	40; 25; 20	20	20	200	0; 5; 0

Fonte: elaborado pelos autores (2026), a partir da saída do script simplex_edepa.py (Python 3.14.4; SciPy 1.18.1, HiGHS).

As restrições de equipe (Equação 2) e de revisão humana (Equação 4) são ativas; a do pipeline (Equação 3) não é. A folga $s_2 = 4$ h corresponde à capacidade ociosa do pipeline, e não a horas de equipe disponíveis: as 40 h da equipe já estão consumidas, de modo que elevar x_1 para 24 h mantendo $x_2 = 20$ h é inviável, e substituir 4 h de revisão por SAST leva a (24, 16), com $Z = 192$, inferior ao ótimo.

Na sensibilidade, 1 h adicional de equipe eleva Z em 4 pontos; 1 h adicional de capacidade de revisão eleva Z em 2 pontos, ao realocar 1 h de SAST para revisão; e 1 h adicional no pipeline não altera Z. Esses ganhos marginais coincidem com os preços-sombra do quadro ótimo (4, 2 e 0) e valem enquanto a base ótima se mantém: $20 \leq b_1 \leq 44$ h, $16 \leq b_3 \leq 40$ h e $b_2 \geq 20$ h. Como o modelo não inclui custos, esses valores não indicam qual expansão é economicamente preferível; essa comparação exigiria o custo de cada hora adicional de equipe, de revisão e de infraestrutura. Assim, não se conclui que os revisores sejam o único gargalo: no cenário hipotético, a capacidade total da equipe e o limite de revisão são simultaneamente ativos.

Verificou-se também que a alocação (20, 20) permanece ótima para quaisquer coeficientes com $0 \leq c_1 \leq c_2$; ela depende, portanto, apenas da hipótese ordinal de que a hora de revisão gera pelo menos tanta redução de risco quanto a hora de SAST, enquanto o valor de Z depende dos valores adotados. A definição das unidades, por sua vez, altera a decisão: com $\alpha = 0{,}5$, por exemplo, a solução passaria a (24, 20), com $Z = 216$. A otimalidade matemática para os parâmetros adotados não constitui validação empírica desses parâmetros nem dos achados de segurança.

CONCLUSÃO

Este trabalho articulou uma revisão narrativa (2019–2026), um ensaio exploratório e um modelo ilustrativo de PL sobre a verificação de código gerado por IAG em DevSecOps, no contexto proposto da camada de supervisão de uma unidade agroindustrial. A literatura indica que código gerado por IA pode conter vulnerabilidades, que LLMs podem apoiar a triagem de alertas de análise estática e que a cadeia de suprimentos de modelos introduz riscos próprios; as métricas dos estudos, porém, não são diretamente comparáveis entre si.

No ensaio, 4 dos 5 trechos receberam ao menos um alerta automático do Bandit, e a revisão manual identificou um caso de *Path Traversal* não sinalizado. Esses resultados ilustram a complementaridade entre análise estática e revisão humana no conjunto testado, mas não estimam taxas de vulnerabilidade, não confirmam individualmente os alertas e não avaliam a segurança de um pipeline em

produção nem de uma instalação agroindustrial.

O modelo de PL, resolvido pelo Método Simplex e verificado computacionalmente, indicou a alocação de 20 h de SAST e 20 h de revisão humana (Z* = 200 pontos no cenário hipotético), com equipe e revisão como restrições ativas e folga de 4 h no pipeline. Trata-se de um instrumento demonstrativo, cuja utilidade decisória depende da calibração dos coeficientes, com registro de tempos e escala de risco definida, e da inclusão de custos. Como trabalhos futuros, propõem-se ampliar o ensaio com mais tarefas, gerações repetidas e modelos identificados; documentar a confirmação humana de cada alerta; calibrar os coeficientes com dados de tempo e de risco; incluir custos de capacidade; e avaliar mecanismos de atestação de modelos na cadeia de suprimentos.

NOMENCLATURA

x_1 – horas semanais da equipe em análise estática automatizada (SAST) [h/semana]
x_2 – horas semanais da equipe em revisão humana de código [h/semana]
Z – índice hipotético de redução de risco (função objetivo) [pontos/semana]
c_1, c_2 – coeficientes de redução de risco por hora de x_1 e de x_2 (hipotéticos: 4 e 6) [pontos/h]
b_1, b_2, b_3 – capacidades de equipe (40), pipeline (24) e revisão humana (20) [h/semana]
s_1, s_2, s_3 – variáveis de folga das restrições de equipe, pipeline e revisão [h/semana]
α – fator de conversão de hora de execução em hora de trabalho humano [adimensional]
* – sobrescrito que indica valor na solução ótima [—]

AST – Application Security Testing
CI/CD – Continuous Integration / Continuous Delivery
CLP – Controlador Lógico Programável
CWE – Common Weakness Enumeration
DevSecOps – Development, Security and Operations
IAG – Inteligência Artificial Generativa
IoT – Internet of Things (Internet das Coisas)
LLM – Large Language Model
OWASP – Open Worldwide Application Security Project
PL – Programação Linear
SAST – Static Application Security Testing
SCADA – Supervisory Control and Data Acquisition
SQL – Structured Query Language
TI / TO – Tecnologia da Informação / Tecnologia Operacional
XML – Extensible Markup Language
XXE – XML External Entity

REFERÊNCIAS

CARLINI, N.; JAGIELSKI, M.; CHOQUETTE-CHOO, C. A.; PALEKA, D.; PEARCE, W.; ANDERSON, H.; TERZIS, A.; THOMAS, K.; TRAMÈR, F. (2024), Poisoning Web-Scale Training Datasets is Practical, Anais do 2024 IEEE Symposium on Security and Privacy (SP), p.407-425, San Francisco - CA. DOI: 10.1109/SP54263.2024.00179.

DU, X.; FENG, J.; ZOU, Y.; XU, W.; MA, J.; ZHANG, W.; LIU, S.; PENG, X.; LOU, Y. (2026), Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry, arXiv preprint arXiv:2601.18844v1, 26 jan. 2026 (não revisado por pares).

FIROUZI, E.; GHAFARI, M. (2026), Can Generative AI Detect and Fix Real-World Cryptographic Misuses?, Journal of Systems and Software, Vol 232, 112650. DOI: 10.1016/j.jss.2025.112650.

HUBINGER, E.; DENISON, C.; MU, J.; LAMBERT, M.; TONG, M.; MACDIARMID, M.; LANHAM, T.; ZIEGLER, D. M.; MAXWELL, T.; CHENG, N.; JERMYN, A.; ASKELL, A.; RADHAKRISHNAN, A.; ANIL, C.; DUVENAUD, D.; GANGULI, D.;

BAREZ, F.; CLARK, J.; NDOUSSE, K.; SACHAN, K.; SELLITTO, M.; SHARMA, M.; DASSARMA, N.; GROSSE, R.; KRAVEC, S.; BAI, Y.; WITTEN, Z.; FAVARO, M.; BRAUNER, J.; KARNOFSKY, H.; CHRISTIANO, P.; BOWMAN, S. R.; GRAHAM, L.; KAPLAN, J.; MINDERMANN, S.; GREENBLATT, R.; SHLEGERIS, B.; SCHIEFER, N.; PEREZ, E. (2024), Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training, arXiv preprint arXiv:2401.05566v3 (não revisado por pares).

MITRE CORPORATION (2026), Common Weakness Enumeration (CWE): CWE-20, CWE-22, CWE-78, CWE-89, CWE-327, CWE-611 e CWE-916. Disponível em: https://cwe.mitre.org/. Acesso em: 29 out. 2026.

OWASP FOUNDATION (2025), LLM03:2025 Supply Chain, OWASP Top 10 for LLM Applications 2025, OWASP Gen AI Security Project. Disponível em: https://genai.owasp.org/llmrisk/llm032025-supply-chain/. Acesso em: 20 ago. 2026.

PEARCE, H.; AHMAD, B.; TAN, B.; DOLAN-GAVITT, B.; KARRI, R. (2021), Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions, arXiv preprint arXiv:2108.09293v3. Versão revisada por pares: Anais do 2022 IEEE Symposium on Security and Privacy (SP), p.754-768, San Francisco - CA, DOI: 10.1109/SP46214.2022.9833571.

PERRY, N.; SRIVASTAVA, M.; KUMAR, D.; BONEH, D. (2023), Do Users Write More Insecure Code with AI Assistants?, Anais da 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23), p.2785-2799, Copenhague. DOI: 10.1145/3576915.3623157.

PINHEIRO, J. M. S. (2019), Segurança Cibernética Integrada na Indústria 4.0, Anais do 23º Seminário de Automação e TI - ABM Week 2019, p.218-227, São Paulo - SP. DOI: 10.5151/2594-5335-33626.

PYCQA (2026), Bandit Documentation: Test Plugins and Blacklist Calls. Disponível em: https://bandit.readthedocs.io/. Acesso em: 29 out. 2026.

PYTHON SOFTWARE FOUNDATION (2026), XML Processing Modules: XML Security, Python 3 Documentation. Disponível em: https://docs.python.org/3/library/xml.html. Acesso em: 29 out. 2026.

TAHA, H. A. (2017), Operations Research: An Introduction, 10. ed., Pearson, Boston.

WANG, J.; LUO, X.; CAO, L.; HE, H.; HUANG, H.; XIE, J.; JATOWT, A.; CAI, Y. (2024), Is Your AI-Generated Code Really Secure? Evaluating Large Language Models on Secure Code Generation with CodeSecEval, arXiv preprint arXiv:2407.02395v1, 2 jul. 2024 (não revisado por pares; versões posteriores do mesmo identificador têm título, autoria e conteúdo distintos).

YANG, S.; ZHENG, X.; XU, J.; LI, J.; SONG, D.; CHEN, Z.; NGAI, E. C. H. (2025), Generative AI for Vulnerability Detection in 6G Wireless Networks: Advances, Case Study, and Future Directions, arXiv preprint arXiv:2506.20488v1 (não revisado por pares).

ZHANG, C.; PAL, R.; NICHOLSON, C.; SIEGEL, M. (2024), (Gen)AI Versus (Gen)AI in Industrial Control Cybersecurity, Anais da 2024 Winter Simulation Conference (WSC), p.2739-2750, Orlando - FL. DOI: 10.1109/WSC63780.2024.10838831.